

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Muhammad Arifin Umasangadji - 5024241083

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler merupakan komponen elektronik yang dapat diprogram untuk mengolah masukan dan mengendalikan perangkat lain. Penggunaannya cukup luas, mulai dari perangkat sederhana seperti lampu otomatis hingga sistem yang melibatkan sensor dan komunikasi data. Oleh karena itu, pemahaman mengenai cara kerja dan pemrograman mikrokontroler menjadi salah satu dasar dalam mempelajari sistem tertanam (*embedded system*).

Pada praktikum ini digunakan dua board, yaitu STM32 Blackpill dan ESP32 DevKit. Keduanya diprogram menggunakan PlatformIO pada Visual Studio Code dengan framework yang sesuai. Melalui kegiatan ini, praktikan mempelajari dasar penggunaan mikrokontroler serta perbedaan lingkungan pengembangan pada kedua board tersebut.

Praktikum juga membahas beberapa fitur dasar mikrokontroler, seperti GPIO untuk membaca input dan mengendalikan output, penggunaan pushbutton, serta pembacaan sensor LDR melalui ADC. Materi tersebut diharapkan dapat membantu praktikan memahami hubungan antara program, mikrokontroler, dan komponen elektronik yang digunakan.

1.2 Dasar Teori

1.2.1 Mikrokontroler dan Sistem Tertanam

Mikrokontroler adalah chip yang dapat menjalankan program untuk mengolah data dan mengendalikan perangkat elektronik. Mikrokontroler biasanya menjadi bagian dari sistem tertanam, yaitu sistem komputer yang dibuat untuk menjalankan fungsi tertentu pada suatu perangkat.

1.2.2 PlatformIO, STM32, dan ESP32

PlatformIO merupakan lingkungan pengembangan untuk menulis, mengompilasi, dan mengunggah program ke board mikrokontroler. PlatformIO dapat digunakan melalui Visual Studio Code dan mendukung berbagai jenis board serta framework. Pada praktikum ini, STM32 Blackpill menggunakan framework Arduino, sedangkan ESP32 DevKit menggunakan framework ESP-IDF.

1.2.3 General Purpose Input/Output (GPIO)

GPIO merupakan pin mikrokontroler yang dapat diatur sebagai input atau output. Pin input digunakan untuk membaca kondisi komponen, misalnya pushbutton, sedangkan pin output digunakan untuk mengendalikan komponen seperti LED. Pada framework Arduino, GPIO dapat diatur dan dibaca menggunakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `digitalRead()`.

1.2.4 Pull-up, Pull-down, dan Debouncing

Pull-up dan pull-down digunakan untuk menjaga kondisi logika pin input agar tetap jelas saat tombol tidak ditekan. Pull-up membuat kondisi awal pin bernilai HIGH, sedangkan pull-down membuatnya bernilai LOW. Sementara itu, debouncing merupakan cara untuk mengatasi sinyal tombol yang tidak stabil ketika ditekan atau dilepas, sehingga satu penekanan tidak terbaca berulang kali.

1.2.5 ADC dan Sensor LDR

ADC (*Analog-to-Digital Converter*) berfungsi mengubah sinyal tegangan analog menjadi nilai digital yang dapat diproses mikrokontroler. Salah satu komponen yang dapat dibaca melalui ADC adalah LDR (*Light Dependent Resistor*). Hambatan LDR berubah mengikuti intensitas cahaya yang diterima, sehingga perubahan tegangan pada rangkaian dapat dibaca oleh mikrokontroler.

2 Tugas Pendahuluan

1. **Perbedaan Framework Arduino dan ESP-IDF**

   Framework Arduino dan ESP-IDF memiliki perbedaan dalam tingkat kemudahan penggunaan dan pengaturan sistem. Arduino menyediakan fungsi yang lebih sederhana sehingga proses pemrograman mikrokontroler menjadi lebih mudah dipahami, terutama bagi pemula. Kelebihannya adalah memiliki banyak library dan contoh program, tetapi pengaturan sistem yang lebih kompleks menjadi lebih terbatas.

   Sementara itu, ESP-IDF merupakan framework resmi dari Espressif untuk pengembangan mikrokontroler ESP32. ESP-IDF memberikan kontrol yang lebih luas terhadap perangkat keras dan fitur ESP32, seperti pengaturan task, komunikasi, serta manajemen memori. Kekurangannya adalah struktur program dan proses konfigurasi relatif lebih kompleks dibandingkan Arduino. Dengan demikian, Arduino lebih mudah digunakan untuk proyek sederhana, sedangkan ESP-IDF lebih sesuai untuk pengembangan yang membutuhkan pengaturan sistem lebih mendalam.

2. **Perbedaan Proses Flashing STM32 Blackpill dan ESP32**

   STM32 Blackpill umumnya memerlukan programmer eksternal seperti ST-Link untuk mengunggah program melalui antarmuka SWD (*Serial Wire Debug*). Hal ini karena ST-Link berfungsi sebagai penghubung antara komputer dan mikrokontroler untuk proses pemrograman maupun debugging. Pada konfigurasi board yang digunakan, konektor USB tidak selalu dilengkapi chip USB-to-Serial sehingga tidak dapat langsung digunakan untuk proses flashing melalui serial.

   Berbeda dengan STM32 Blackpill, board ESP32 DevKit umumnya sudah dilengkapi chip USB-to-Serial, seperti CP2102 atau CH340. Chip tersebut berfungsi mengubah komunikasi USB dari komputer menjadi komunikasi serial UART yang dapat diterima oleh ESP32. Dengan adanya chip ini dan dukungan bootloader pada ESP32, proses pengunggahan program dapat dilakukan langsung menggunakan kabel USB tanpa memerlukan programmer eksternal.

3. **Perbedaan Konfigurasi Pull-up dan Pull-down**

   Konfigurasi pull-up dan pull-down digunakan untuk menjaga kondisi logika pin input agar tidak mengambang (*floating*) ketika tombol tidak ditekan. Perbedaan keduanya terletak pada kondisi logika awal dan perubahan kondisi saat tombol ditekan.

   Pada konfigurasi pull-up, pin input dihubungkan ke tegangan VCC melalui resistor. Ketika tombol tidak ditekan, pin akan bernilai HIGH. Saat tombol ditekan, pin terhubung ke GND sehingga bernilai LOW.

   Sebaliknya, pada konfigurasi pull-down, pin input dihubungkan ke GND melalui resistor. Ketika tombol tidak ditekan, pin akan bernilai LOW. Saat tombol ditekan, pin terhubung ke VCC sehingga bernilai HIGH.

4. **Fenomena Bouncing pada Tombol Mekanik**

   Bouncing merupakan fenomena ketika kontak logam pada tombol mekanik mengalami sentuhan dan pelepasan secara berulang dalam waktu yang sangat singkat saat tombol ditekan atau dilepaskan. Hal ini terjadi karena adanya getaran dan pantulan fisik pada bagian kontak tombol sebelum akhirnya berada pada kondisi stabil.

   Fenomena tersebut dapat menyebabkan mikrokontroler membaca satu kali penekanan tombol sebagai beberapa kali input. Untuk mengatasinya, terdapat beberapa pendekatan, yaitu *software debouncing* dengan memberikan jeda waktu menggunakan delay atau timer sebelum membaca kondisi tombol kembali, serta *hardware debouncing* dengan menggunakan rangkaian resistor dan kapasitor untuk menstabilkan sinyal input.

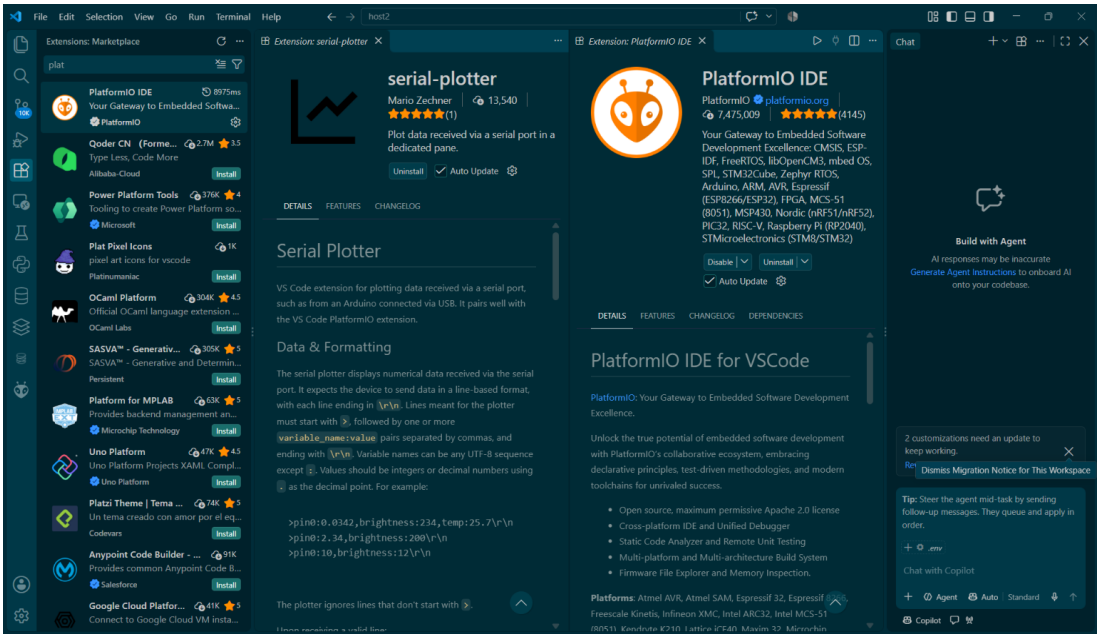
5. **Cara Kerja ADC dan Pengaruh Resolusi Bit**

   ADC (*Analog-to-Digital Converter*) merupakan komponen yang berfungsi mengubah sinyal analog menjadi nilai digital agar dapat diproses oleh mikrokontroler. Proses ini dilakukan dengan membaca tegangan analog, kemudian mengubahnya menjadi nilai digital berdasarkan rentang tegangan referensi yang digunakan.

   Resolusi ADC menentukan jumlah kemungkinan nilai digital yang dapat dihasilkan. Sebagai contoh, ADC dengan resolusi 10-bit memiliki 2^{10} atau 1024 tingkat nilai, sedangkan ADC 12-bit memiliki 2^{12} atau 4096 tingkat nilai. Semakin tinggi resolusi ADC, semakin kecil perubahan tegangan yang dapat dibedakan sehingga hasil pembacaan menjadi lebih rinci. Namun, ketelitian pengukuran juga dipengaruhi oleh faktor lain seperti kestabilan tegangan referensi dan karakteristik ADC.

6. **Setup Tools Praktikum**

   Berikut dokumentasi hasil setup tools pada laptop:



Gambar 1: Tampilan Visual Studio Code dan ekstensi yang telah terpasang